COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	October 13, 2026	**DATE SUBMITTED:**	October 14, 2026
NAME/S:	De Leon, Benj Federic C. Mendoza, Ryo Shin P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 ALGORITHMS STRATEGIES

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review).

Output:

Strategy	Algorithm	Analysis
Recursion	Fibonacci Sequence	The algorithm uses recursion by calling itself to calculate the previous two Fibonacci numbers until it reaches the base cases.
Brute Force	Linear Search	The algorithm checks each element one by one until the target value is found or all elements have been checked.
Backtracking	Depth-First Search	The algorithm explores a path as far as possible and returns to a previous node when it reaches a dead end or has no unvisited neighbors.
Greedy	Selection Sort	The algorithm selects the smallest element from the unsorted portion in each pass and places it in its correct position. This demonstrates choosing the smallest available element at each step, although Selection Sort is not a standard greedy algorithm.
Divide-and-Conquer	Merge Sort	The algorithm divides the array into smaller subarrays, sorts them recursively, and merges them to produce the final sorted array.

Table 12-1. Algorithmic Strategies and Examples

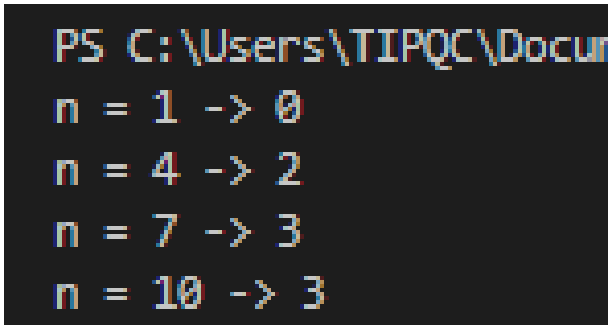
Screenshots	 
Analysis	It uses top-down recursion and saves each result in memo[]. Each subproblem is solved once, so it avoids repeated work. For n = 10 it finds 10→9→3→1 (3 steps), which greedy misses.

Table 12-2. Memoization Implementation

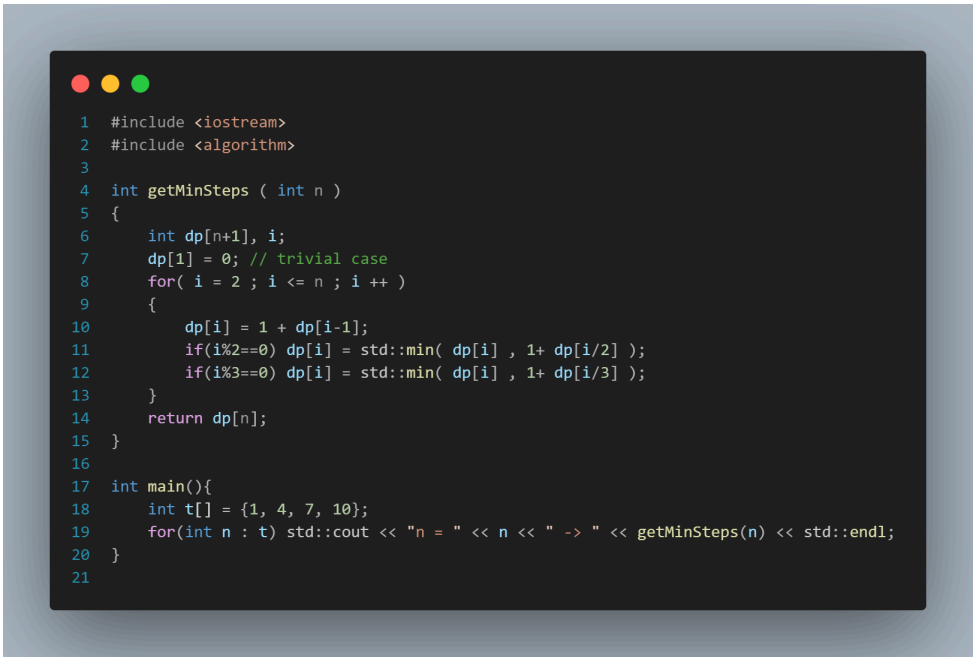
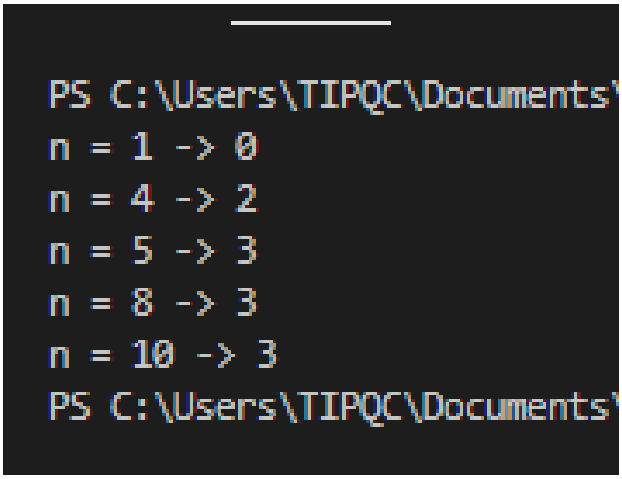
Screenshots	 
Analysis	It starts from dp[1] and fills the table up to n, so every subproblem is solved before it is needed. It has no recursion, so there is no stack-overflow risk.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Pseudocode

```
START

INPUT matrix, M, N, targetCost

CREATE a DP table dp[M][N]
INITIALIZE all values in dp to 0

FOR each row i from 0 to M-1
   FOR each column j from 0 to N-1
      IF i = 0 AND j = 0 THEN
         dp[i][j][matrix[i][j]] = 1
      ELSE
         FOR each possible cost c
            IF moving from above is possible THEN
               ADD paths from dp[i-1][j][c - matrix[i][j]]
            END IF

            IF moving from left is possible THEN
               ADD paths from dp[i][j-1][c - matrix[i][j]]
            END IF
         END FOR
      END IF
   END FOR
END FOR

OUTPUT number of paths reaching the destination
      with total cost equal to targetCost

END
```

Solving by hand

Target cost: 25

Step 1: Understand the movement.

- Start at (0,0), whose cost is 4.
- Move only right or down.
- Reach (3,3), whose cost is 3.

- Add the costs of all visited cells.

Step 2: Calculate the path costs.

For example, the path going right and then down can be represented as:

$$(0, 0) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (1, 2) \rightarrow (2, 2) \rightarrow (3, 2) \rightarrow (3, 3)$$

Its total cost is: 4+7+1+3+1+7+3= **26**

Another path is:

$$(0, 0) \rightarrow (1, 0) \rightarrow (1, 1) \rightarrow (1, 2) \rightarrow (2, 2) \rightarrow (2, 3) \rightarrow (3, 3)$$

Its total cost is: 4+6+7+3+1+2+3= **26**

Note: The matrix shown in the problem has two paths with a total cost of **26**, not 25. For the displayed matrix, there are zero paths with a total cost of exactly **25**.

Based on the highlighted path shown in the matrix, the path has a total of 21, with one path.

4+7+1+3+1+2+3 = 21

Code

```cpp
#include <iostream>
#include <vector>

int countPaths(const std::vector<std::vector<int>>& matrix,
               int targetCost) {
    int rows = matrix.size();
    int cols = matrix[0].size();

    // dp[i][j][cost] = number of paths reaching cell (i, j)
    // with the given total cost.
    std::vector<std::vector<std::vector<long long>>> dp(
        rows,
        std::vector<std::vector<long long>>(
            cols, std::vector<long long>(targetCost + 1, 0)
        )
    );

    // Initialize the starting cell.
    if (matrix[0][0] <= targetCost) {
        dp[0][0][matrix[0][0]] = 1;
    }

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (i == 0 && j == 0) {
                continue;
            }

            int cellCost = matrix[i][j];
```

```
            // A path cannot exceed the target cost.
            if (cellCost > targetCost) {
                continue;
            }

            for (int cost = cellCost;
                 cost <= targetCost; cost++) {

                int previousCost = cost - cellCost;

                // Move down from the cell above.
                if (i > 0) {
                    dp[i][j][cost] +=
                        dp[i - 1][j][previousCost];
                }

                // Move right from the cell on the left.
                if (j > 0) {
                    dp[i][j][cost] +=
                        dp[i][j - 1][previousCost];
                }
            }
        }
    }

    return dp[rows - 1][cols - 1][targetCost];
}

int main() {
    std::vector<std::vector<int>> matrix = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int targetCost;

    std::cout << "Enter the target cost: ";
    std::cin >> targetCost;

    int result = countPaths(matrix, targetCost);

    std::cout << "Number of paths with cost "
              << targetCost << ": " << result << '\n';

    return 0;
}
```

Analysis

The program uses Dynamic Programming to count the number of paths that reach the destination cell with the given total cost. It only allows movement to the right or down. The program stores previously calculated results in a DP table to avoid repeating calculations.

The highlighted path has a total cost of $4 + 7 + 1 + 3 + 1 + 2 + 3 = 21$. The program checks all possible valid paths and counts only those whose total cost matches the target cost. But based on our solution by hand, the total cost of two paths is not 25, but 26.

The time complexity is $O(MNK)$, while the space complexity is $O(MNK)$, where M is the number of rows, N is the number of columns, and K is the target cost. Overall, Dynamic Programming makes the solution more efficient by reusing previously calculated results.

Demonstration and Output

```
Enter the target cost: 21
Number of paths with cost 21: 1
```

This is the correct result for the highlighted matrix provided.

```
Enter the target cost: 26
Number of paths with cost 26: 2
```

This matches the two paths calculated by hand.

```
Enter the target cost: 25
Number of paths with cost 25: 0
```

This is the incorrect result based on the problem, since there is no valid path in the given matrix that has a total cost of 25.

Conclusion:

De Leon, Benj Federic	I learned how recursion, brute force, backtracking, greedy, divide-and-conquer and dynamic programming differ. Greedy fails on “minimum steps to one”, while memoization and bottom-up DP both solve it correctly. The same recursive,

	optimal-substructure idea also solved the matrix path-counting problem.
Mendoza, Ryo	In this activity, we learned how to use Dynamic Programming to count the number of paths in a matrix with a specific total cost. We understood how the program stores previous results to avoid repeating calculations and only allows movement to the right or down. Based on our output, the highlighted path has a total cost of 21, while no valid path has a total cost of exactly 25. Overall, this activity helped us understand how Dynamic Programming can solve problems more efficiently by reusing previously calculated results.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Claude

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance and explanation of concepts: writing and testing C++ code, and drafting the analysis and conclusion for the DSA activity.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Claude was used to produce and run the C++ solutions for the memoization, bottom-up DP and path-counting tasks, and to draft the analysis and conclusion. We reviewed the output, checked that the programs ran, and adjusted the answers before submitting.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.